# Übungsblatt 4: Aktorik & Sicherheit

**Mikrocomputertechnik 3 (MCT3)**

## Zielsetzung

Sie vertiefen Vorlesung 4: Hardware-Timer für PWM (PSC, ARR, CCR), Duty Cycle und mittlere Spannung, Software- vs. Hardware-PWM sowie Watchdog-/Window-Watchdog-Sicherheit — inklusive kurzer C-Praxis.

**Bearbeitungszeit:** ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 14 P, Aufgabe 2: 8 P (Gesamt: 22 P)

## Teil 1: Theorie — Hardware-Counter, PWM und Watchdog

### Aufgabe 1.1: Komponenten des Hardware-Timers

PWM wird in Hardware oft über General-Purpose-Timer (GPT) erzeugt.

1. Erklären Sie die Funktion von Prescaler (`PSC`), Auto-Reload Register (`ARR`) und Capture/Compare Register (`CCR`) bei der PWM-Erzeugung.
2. Erläutern Sie anhand von Zählerstand (`CNT`) und `CCR`, wie bei Edge-Aligned PWM (Active-High) HIGH und LOW entstehen.

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 1.2: Mathematik der PWM-Erzeugung

Ein SoC läuft mit $f_{\text{clock}} = 100\,\text{MHz}$. Timer: `PSC = 99`, `ARR = 1000`, `CCR = 250`. Pin: 0 V / 3,3 V.

1. Berechnen Sie die Zählfrequenz $f_{\text{cnt}}$. Formel angeben.
2. Bestimmen Sie den Duty Cycle in Prozent (vereinfachtes Modell CCR/ARR).
3. Nähern Sie die mittlere Spannung $V_{\text{mittel}}$ am Pin (Vorlesungsmodell: Mittelwert $\approx V_{\text{Pin}} \times$ Duty Cycle).

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 1.3: Software-PWM vs. Hardware-PWM

1. Erklären Sie Bit-Banging (Software-PWM) und nennen Sie zwei Nachteile gegenüber Hardware-PWM.

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 1.4: Watchdog Timer (WDT)

1. Was ist ein Watchdog Timer, und was passiert hardwareseitig bei Zählerstand 0?
2. Warum ist das Füttern des WDT in einer periodischen Timer-ISR (z. B. SysTick) gefährlich? Wo sollte `wdt_feed()` stattdessen sitzen?
3. Welches zusätzliche Problem adressiert ein Window Watchdog (WWDG)?

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

## Teil 2: Praxis — PWM und Watchdog in C

Adressen sind didaktisch/fiktiv.

### Aufgabe 2.1: LED-Helligkeit per Hardware-PWM

`TIMER_ARR = 1000`. Helligkeit als Prozent 0…100.

```c
#include <stdint.h>

#define TIMER_BASE 0x40000000u
#define TIMER_ARR  (*(volatile uint32_t *)(TIMER_BASE + 0x04u))
#define TIMER_CCR1 (*(volatile uint32_t *)(TIMER_BASE + 0x0Cu))

void set_led_brightness(uint8_t percent) {
    /* 1. percent auf max. 100 begrenzen */
    /* TODO */

    /* 2. CCR aus Prozent und TIMER_ARR berechnen */
    /* TODO */

    /* 3. in TIMER_CCR1 schreiben */
    /* TODO */
}
```

#### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*

### Aufgabe 2.2: Freeze provozieren und WDT-Heilung

Simulieren Sie einen Deadlock per Tastendruck. Nach WDT-Reset soll die Ursache auf der Konsole erscheinen. Füttern Sie den WDT an der architektonisch richtigen Stelle.

```
#include <stdio.h>
#include <stdint.h>

#define RESET_REASON_REG (*(volatile uint32_t *)0x40020000u)
#define WDT_RESET_FLAG   (1u << 2)

void check_reset_reason(void) {
    /* TODO: WDT_RESET_FLAG prüfen und ggf. warnen */
}

int main(void) {
    check_reset_reason();
    init_pwm();
    init_wdt(); /* Timeout z. B. 2 s */

    while (1) {
        if (button_pressed()) {
            printf("Freeze provoziert! Warte auf WDT...\n");
            /* TODO: künstlicher Deadlock */
        }

        set_led_brightness(50);
        /* TODO: wdt_feed() an korrekter Stelle */
    }
}
```

### Ihre Lösung

*(Notieren Sie Ihre Lösung hier.)*